

ITAI 4374

Neuroscience as a Model for AI

MODULE 02 LAB

Cortical Column Voting Simulation

Student Lab Guide

Estimated Time: 2-3 hours

Platforms: Python (Google Colab) or Wolfram
Language

Prerequisites: Module 02 Booklet

WARNING

This guide provides starter code with sections YOU must complete.

Look for  YOUR TASK boxes — these indicate code you need to write.

Do not skip ahead! Each section builds on previous work.

Part 1: Introduction

1.1 What You Will Build

In this lab, you will implement a simplified simulation of cortical column voting based on Jeff Hawkins' Thousand Brains Theory. Your simulation will demonstrate:

- Multiple "columns" that each receive partial sensory input
- A voting mechanism where columns share beliefs
- Consensus emerging from distributed processing
- Visualization of how agreement develops over time

1.2 The Algorithm

Your simulation will follow these steps:

Step	What Happens	What You'll Code
1. Setup	Create N columns, define possible objects	Basic initialization
2. Sensory Input	Each column receives partial, noisy evidence	<code>create_sensory_input()</code>
3. Initialize	Set initial beliefs from evidence	<code>initialize_beliefs()</code>
4. Vote	Columns share beliefs with neighbors	<code>voting_round()</code>
5. Iterate	Repeat voting until consensus	<code>run_voting()</code>
6. Visualize	Show how consensus emerges	<code>visualize_consensus()</code>

1.3 Key Concepts to Understand

Before coding, make sure you understand these concepts from **Module 02**:

- **Distributed Representation:** No single column knows the answer; knowledge emerges from many columns working together.
- **Lateral Connections:** Columns communicate with neighbors, sharing their current beliefs.
- **Consensus:** Through iterative voting, columns converge on a shared interpretation.
- **Noise/Ambiguity:** Each column receives imperfect evidence, creating uncertainty.

1.4 Learning Goals

By completing this lab, you will:

1. Implement a voting algorithm from scratch
2. Understand how distributed systems reach consensus
3. Explore trade-offs between speed and accuracy
4. Connect abstract theory to working code
5. Identify limitations of simplified models

Part 2: Environment Setup

2.1 Choose Your Platform

Choose ONE: Python (recommended) or Wolfram Language.

Python (Google Colab)	Wolfram Language
Free, no installation	College subscription required
Go to colab.research.google.com	Use Mathematica or wolframcloud.com
Click "New Notebook"	Create new notebook
NumPy and Matplotlib included	All functions built-in

2.2 Python Setup

In your first Colab cell, run this setup code:

Python

```
# Setup - Run this first!
import numpy as np
import matplotlib.pyplot as plt

# Set seed for reproducibility
np.random.seed(42)

# Define objects our system can recognize
OBJECTS = ["cup", "ball", "book", "phone", "apple"]
NUM_OBJECTS = len(OBJECTS)

print("Setup complete!")
print("Objects:", OBJECTS)
```

2.3 Wolfram Setup

In your first Wolfram cell, run:

Wolfram

```
(* Setup - Run this first! *)
objects = {"cup", "ball", "book", "phone", "apple"};
numObjects = Length[objects];
SeedRandom[42];

Print["Setup complete!"]
Print["Objects: ", objects]
```

☒ CHECKPOINT

Verify you see: Setup complete!

Verify you see the list of 5 objects

If you get errors, check spelling and run again

Part 3: Implementation Tasks

This section contains the functions you need to implement. Starter code is provided, but you must complete the TODO sections.

Task 1: Create Sensory Input

This function simulates what each cortical column "senses." Each column receives different, partial evidence about the object.

What the function should do:

6. Create a matrix of shape (num_columns, NUM_OBJECTS)
7. Fill it with random noise between 0 and noise_level
8. Add a stronger signal (0.5 to 1.0) for the TRUE object in each column
9. Normalize each row so values sum to 1 (like probabilities)
10. Return the evidence matrix

Python Starter Code:

Python — STARTER CODE

```
def create_sensory_input(true_object, num_columns, noise_level=0.3):
    """
    Creates evidence matrix for all columns.

    Parameters:
    - true_object: int, index of the actual object (0 to 4)
    - num_columns: int, number of columns to simulate
    - noise_level: float, amount of random noise (0 to 1)

    Returns:
    - evidence: numpy array of shape (num_columns, NUM_OBJECTS)
    """

    # TODO 1: Create matrix filled with random noise
    # Hint: Use np.random.uniform(low, high, size)
    # Size should be (num_columns, NUM_OBJECTS)
    evidence = _____ # YOUR CODE HERE

    # TODO 2: Add signal for the true object
    # Loop through each column and add random value between 0.5-1.0
    # to the true_object index
    for col in range(num_columns):
        signal = _____ # YOUR CODE: random value 0.5 to 1.0
        evidence[col, true_object] += signal

    # TODO 3: Normalize each row to sum to 1
    # Hint: Divide each row by its sum
    # Use axis=1 and keepdims=True
    row_sums = _____ # YOUR CODE HERE
    evidence = evidence / row_sums

    return evidence
```

Wolfram Starter Code:

Wolfram — STARTER CODE

```
createSensoryInput[trueObject_, numColumns_, noiseLevel_: 0.3] :=
  Module[{evidence, signal},

    (* TODO 1: Create matrix with random noise *)
    (* Hint: Use Table and RandomReal *)
    evidence = Table[
      _____, (* YOUR CODE: random list of length numObjects *)
      {numColumns}
    ];

    (* TODO 2: Add signal for true object *)
    Do[
      signal = _____; (* YOUR CODE: random 0.5 to 1.0 *)
      evidence[[col, trueObject]] += signal,
      {col, numColumns}
    ];

    (* TODO 3: Normalize each row *)
    (* Hint: Map a function that divides by Total *)
    evidence = _____; (* YOUR CODE HERE *)

    evidence
  ]
```

YOUR TASK

Complete the three TODOs in the function above

Test your function with the test code below

Your output should show each column with different confidence levels

Test Your Function:

Python

```
# Test create_sensory_input
true_object = 0 # "cup"
num_columns = 6

evidence = create_sensory_input(true_object, num_columns, noise_level=0.3)

print(f"True object: {OBJECTS[true_object]}")
print(f"Evidence shape: {evidence.shape}")
print("\nEach column's evidence:")
for i in range(num_columns):
    best = OBJECTS[np.argmax(evidence[i])]
    conf = np.max(evidence[i]) * 100
    print(f"Column {i}: {best} ({conf:.1f}%)")
```

Expected Output (values will vary):

True object: cup

Evidence shape: (6, 5)

Each column's evidence:

Column 0: cup (42.3%)

Column 1: cup (38.7%)

... (all should say cup with varying confidence)

TIP

If your columns don't all pick 'cup', check that you're adding signal to `evidence[col, true_object]`, not a different index.

Task 2: Initialize Beliefs

This simple function sets each column's initial beliefs equal to its sensory evidence.

What the function should do:

- Take the evidence matrix as input
- Return a COPY of it (so we don't modify the original)

YOUR TASK

This one is simple! Just return a copy of the evidence.

Python: Use `.copy()` method

Wolfram: The function body is just the input (Wolfram copies by default)

Python:

Python — STARTER CODE

```
def initialize_beliefs(evidence):
    """Initialize beliefs from sensory evidence."""
    return _____ # YOUR CODE: return a copy of evidence
```

Wolfram:

Wolfram — STARTER CODE

```
initializeBeliefs[evidence_] := _____ (* YOUR CODE *)
```

Test:

Python

```
beliefs = initialize_beliefs(evidence)
print("Beliefs initialized!")
print(f"Shape: {beliefs.shape}")
print(f"Same as evidence? {np.allclose(beliefs, evidence)}") # Should be True
```

Task 3: The Voting Mechanism

This is the core of the simulation! Each column updates its beliefs by combining its own evidence with votes from other columns.

The Formula:

For each column, the new belief is:

$$\text{new_belief} = (1 - \text{vote_weight}) \times \text{own_evidence} + \text{vote_weight} \times \text{average_neighbor_votes}$$

- **vote_weight = 0.3** means 70% trust in own evidence, 30% trust in neighbors
- **average_neighbor_votes** = mean of all OTHER columns' current beliefs

What the function should do:

11. Create an empty matrix for new beliefs
12. For each column:
 - Calculate own contribution: $(1 - \text{vote_weight}) \times \text{evidence}[\text{col}]$
 - Get votes from other columns (exclude current column)
 - Calculate vote contribution: $\text{vote_weight} \times \text{mean}(\text{other_beliefs})$
 - Add them together
13. Normalize each row to sum to 1
14. Return the new beliefs

Python Starter Code:

Python — STARTER CODE

```
def voting_round(beliefs, evidence, vote_weight=0.3):
    """
    One round of voting among columns.

    Parameters:
    - beliefs: current beliefs, shape (num_columns, NUM_OBJECTS)
    - evidence: original sensory evidence (doesn't change)
    - vote_weight: float 0-1, how much to trust neighbors

    Returns:
    - new_beliefs: updated beliefs after voting
    """
    num_columns = beliefs.shape[0]
    new_beliefs = np.zeros_like(beliefs)

    for col in range(num_columns):
        # TODO 1: Calculate this column's own contribution
        # Hint: (1 - vote_weight) * evidence[col]
        own_contribution = _____ # YOUR CODE

        # TODO 2: Get beliefs from OTHER columns (not this one)
        # Hint: Use list comprehension to get indices != col
        # Then index into beliefs
        other_indices = [i for i in range(num_columns) if i != col]
        other_beliefs = _____ # YOUR CODE: beliefs[other_indices]

        # TODO 3: Calculate average vote from others
        # Hint: Use .mean(axis=0) to average across columns
        avg_vote = _____ # YOUR CODE

        # TODO 4: Calculate vote contribution
        vote_contribution = _____ # YOUR CODE

        # TODO 5: Combine own + vote contributions
        new_beliefs[col] = _____ # YOUR CODE

    # TODO 6: Normalize rows
    row_sums = new_beliefs.sum(axis=1, keepdims=True)
    new_beliefs = _____ # YOUR CODE

    return new_beliefs
```

Wolfram Starter Code:

Wolfram — STARTER CODE

```
votingRound[beliefs_, evidence_, voteWeight_: 0.3] :=
  Module[{numCols, newBeliefs, otherBeliefs, avgVote},

    numCols = Length[beliefs];
    newBeliefs = Table[0.0, {numCols}, {numObjects}];

    Do[
      (* TODO 1: Own contribution *)
      ownContrib = _____ ; (* YOUR CODE *)

      (* TODO 2: Get other columns' beliefs *)
      (* Hint: Use Delete[beliefs, col] *)
      otherBeliefs = _____ ; (* YOUR CODE *)

      (* TODO 3: Average vote *)
      avgVote = _____ ; (* YOUR CODE: Mean[otherBeliefs] *)

      (* TODO 4-5: Combine *)
      newBeliefs[[col]] = _____ ; (* YOUR CODE *)

      , {col, numCols}
    ];

    (* TODO 6: Normalize *)
    newBeliefs = Map[_____, newBeliefs]; (* YOUR CODE *)

    newBeliefs
  ]
```

Test Your Function:

Python

```
# Test voting_round
print("Before voting:")
for i in range(3):
    print(f"  Col {i}: {OBJECTS[np.argmax(beliefs[i])]}
    ({np.max(beliefs[i])*100:.1f}%)")

new_beliefs = voting_round(beliefs, evidence, vote_weight=0.3)

print("\nAfter 1 voting round:")
for i in range(3):
    print(f"  Col {i}: {OBJECTS[np.argmax(new_beliefs[i])]}
    ({np.max(new_beliefs[i])*100:.1f}%)")
```

💡 TIP

After voting, confidence should be MORE similar across columns (they're sharing information).

If confidence goes to 0% or 100%, check your normalization.

Task 4: Run Until Consensus

This function runs multiple voting rounds until columns agree or we hit the maximum iterations.

What the function should do:

15. Initialize beliefs from evidence
16. Store beliefs in a history list (for visualization later)
17. Loop up to max_iterations times:
 - Call voting_round() to update beliefs
 - Add new beliefs to history
 - Check if consensus is reached ($\geq 90\%$ of columns agree)
 - If consensus reached, break out of loop
18. Calculate final result (which object won, with what confidence)
19. Return history, winning object, confidence, and iteration count

How to Check Consensus:

For each column, find which object has the highest belief (argmax). Then count how many columns agree on the most common choice.

Python

```
# Example: checking consensus
predictions = np.argmax(beliefs, axis=1) # Best guess per column
# predictions might be [0, 0, 0, 2, 0, 0] - most say object 0

# Count votes for each object
vote_counts = np.bincount(predictions, minlength=NUM_OBJECTS)
# vote_counts = [5, 0, 1, 0, 0] - object 0 has 5 votes

# Find most common and calculate agreement
most_common = vote_counts.argmax() # = 0
agreement = (predictions == most_common).mean() # = 5/6 = 0.833
```

Python Starter Code:

Python — STARTER CODE

```
def run_voting(evidence, max_iterations=20, vote_weight=0.3,
              consensus_threshold=0.9):
    """
    Run voting until consensus or max iterations.

    Returns: (history, final_object, final_confidence, iterations_used)
    """
    # TODO 1: Initialize beliefs
    beliefs = _____ # YOUR CODE

    # TODO 2: Start history list with initial beliefs
    history = [beliefs.copy()]

    for iteration in range(max_iterations):
        # TODO 3: Do one voting round
        beliefs = _____ # YOUR CODE

        # TODO 4: Add to history
        history.append(_____) # YOUR CODE

        # TODO 5: Check consensus
        predictions = np.argmax(beliefs, axis=1)
        vote_counts = np.bincount(predictions, minlength=NUM_OBJECTS)
        most_common = vote_counts.argmax()
        agreement = _____ # YOUR CODE: fraction that agree

        # TODO 6: Break if consensus reached
        if agreement >= consensus_threshold:
            print(f"Consensus at iteration {iteration + 1}!")
            break

    # Calculate final result
    final_votes = beliefs.mean(axis=0) # Average across columns
    final_object = _____ # YOUR CODE: which object won?
    final_confidence = _____ # YOUR CODE: confidence in winner
    iterations_used = len(history) - 1

    return history, final_object, final_confidence, iterations_used
```

Test Your Function:

Python

```
# Test run_voting
np.random.seed(42)
evidence = create_sensory_input(true_object=0, num_columns=6,
noise_level=0.3)

history, result, confidence, iters = run_voting(evidence)

print(f"Result: {OBJECTS[result]}")
print(f"Confidence: {confidence*100:.1f}%")
print(f"Iterations: {iters}")
print(f"Correct? {'Yes!' if result == 0 else 'No!'})")
```

Expected: Should identify 'cup' with high confidence in just a few iterations.

Task 5: Visualization

Create a visualization showing how consensus emerges over time.

What to Plot:

Plot 1 (Left): Confidence over time for each object

- X-axis: Voting round (0, 1, 2, ...)
- Y-axis: Average confidence across all columns
- One line per object
- Make the TRUE object's line bold/solid, others dashed

Plot 2 (Right): Agreement over time

- X-axis: Voting round
- Y-axis: Fraction of columns that agree
- Draw a horizontal line at 0.9 (consensus threshold)

Python Starter Code:

Python — STARTER CODE

```
def visualize_consensus(history, true_object):
    """Visualize how consensus emerges."""

    # Calculate average confidence per object at each iteration
    # history is a list of belief matrices
    # We need: for each time step, average confidence for each object
    avg_confidence = np.array([h.mean(axis=0) for h in history])
    # Shape: (num_iterations, NUM_OBJECTS)

    # TODO 1: Calculate agreement at each time step
    agreement_over_time = []
    for h in history:
        predictions = np.argmax(h, axis=1)
        counts = np.bincount(predictions, minlength=NUM_OBJECTS)
        agreement = _____ # YOUR CODE: max votes / total columns
        agreement_over_time.append(agreement)

    # Create figure with 2 subplots
    fig, axes = plt.subplots(1, 2, figsize=(12, 4))

    # TODO 2: Plot 1 - Confidence over time
    ax1 = axes[0]
    for obj_idx in range(NUM_OBJECTS):
        # Make true object bold, others dashed
        if obj_idx == true_object:
            style = '-'
            width = 3
        else:
            style = '--'
            width = 1
        ax1.plot(avg_confidence[:, obj_idx], style,
                 linewidth=width, label=OBJECTS[obj_idx])

    ax1.set_xlabel('Voting Round')
    ax1.set_ylabel('Average Confidence')
    ax1.set_title('Confidence Evolution')
    ax1.legend()
    ax1.set_ylim(0, 1)
    ax1.grid(True, alpha=0.3)

    # TODO 3: Plot 2 - Agreement over time
    ax2 = axes[1]
    ax2.plot(agreement_over_time, 'b-', linewidth=2, marker='o')
    ax2.axhline(y=0.9, color='r', linestyle='--', label='Threshold')
    ax2.set_xlabel('Voting Round')
    ax2.set_ylabel('Column Agreement')
    ax2.set_title('Consensus Emergence')
    ax2.legend()
    ax2.set_ylim(0, 1.05)
    ax2.grid(True, alpha=0.3)
```



```
plt.tight_layout()
plt.savefig('consensus_plot.png', dpi=150)
plt.show()

print("Plot saved as consensus_plot.png")
```

YOUR TASK

Complete TODO 1: Calculate agreement as $\max(\text{counts}) / \text{len}(\text{predictions})$

The plotting code (TODO 2 and 3) is mostly done — just run it

You should see confidence converging and agreement rising to meet the threshold

Test Your Visualization:

Python

```
# Create visualization
visualize_consensus(history, true_object=0)
```

CHECKPOINT

You should see two plots side by side

Left: The true object (cup) line rises while others fall

Right: Agreement line rises and crosses the red threshold

If plots look wrong, check your history and agreement calculations

Part 4: Required Experiments

Complete all three experiments below. Record your observations for the Analysis section.

Experiment 1: High Noise

Question: What happens when sensory input is very noisy?

YOUR TASK

Run your simulation with `noise_level = 0.8` (instead of 0.3)

Compare: accuracy, confidence, and number of iterations

Create a visualization for this experiment

Code to run:

Python

```
# EXPERIMENT 1: High Noise
print("="*50)
print("EXPERIMENT 1: HIGH NOISE")
print("="*50)

np.random.seed(42) # Reset seed for fair comparison
evidence_noisy = create_sensory_input(
    true_object=0,
    num_columns=6,
    noise_level=0.8 # <-- Changed from 0.3
)

history_noisy, result_noisy, conf_noisy, iters_noisy =
run_voting(evidence_noisy)

print(f"Result: {OBJECTS[result_noisy]}")
print(f"Confidence: {conf_noisy*100:.1f}%")
print(f"Iterations: {iters_noisy}")
print(f"Correct? {'Yes' if result_noisy == 0 else 'NO - WRONG ANSWER'}")

# Visualize
visualize_consensus(history_noisy, true_object=0)
```

Record Your Observations:

- Was the result correct? _____
- How did confidence compare to the original (0.3 noise)? _____
- Did it take more/fewer iterations? _____
- How did the visualization look different? _____

Experiment 2: More Columns

Question: Does having more columns improve results?

YOUR TASK

Run your simulation with 20 columns (instead of 6)

Compare: accuracy, confidence, and iterations to original

Think about: why might more columns help?

Code to run:

Python

```
# EXPERIMENT 2: More Columns
print("="*50)
print("EXPERIMENT 2: MORE COLUMNS")
print("="*50)

np.random.seed(42)
evidence_many = create_sensory_input(
    true_object=0,
    num_columns=20, # <-- Changed from 6
    noise_level=0.3
)

history_many, result_many, conf_many, iters_many =
run_voting(evidence_many)

print(f"Result: {OBJECTS[result_many]}")
print(f"Confidence: {conf_many*100:.1f}%")
print(f"Iterations: {iters_many}")

print("\nComparison to original (6 columns):")
print(f"  6 columns: {confidence*100:.1f}% confidence")
print(f"  20 columns: {conf_many*100:.1f}% confidence")

visualize_consensus(history_many, true_object=0)
```

Record Your Observations:

- Was confidence higher with more columns? _____
- Did it converge faster or slower? _____
- The brain has ~150,000 columns. What might be the advantage?

Experiment 3: Vote Weight Comparison

Question: How does vote_weight affect the results?

 YOUR TASK

Test vote_weight values: 0.1, 0.3, 0.5, 0.7, 0.9

Record accuracy, confidence, and iterations for each

Think about: what does each extreme mean?

Code to run:**Python**

```
# EXPERIMENT 3: Vote Weight Comparison
print("="*50)
print("EXPERIMENT 3: VOTE WEIGHT")
print("="*50)

weights_to_test = [0.1, 0.3, 0.5, 0.7, 0.9]
results = []

for w in weights_to_test:
    np.random.seed(42) # Same starting point each time
    ev = create_sensory_input(0, 6, noise_level=0.4)
    _, res, conf, iters = run_voting(ev, vote_weight=w)
    correct = "Yes" if res == 0 else "No"
    results.append((w, OBJECTS[res], conf, iters, correct))

print(f"{'Weight':<8} {'Result':<8} {'Conf':<10} {'Iters':<8} {'Correct':<8}")
print("-"*45)
for w, obj, conf, iters, correct in results:
    print(f"{w:<8} {obj:<8} {conf*100:>5.1f}% {iters:<8} {correct}")
```

Record Your Observations:

- Which weight gave the best results? _____
- What happened with weight = 0.1 (barely listening to neighbors)?

- What happened with weight = 0.9 (almost ignoring own evidence)?

- What trade-off is vote_weight balancing? _____

Part 5: Analysis & Reflection

Answer these questions based on your experiments. Include your answers in your notebook or submit as a separate document (L02a_...).

5.1 Experiment Analysis

Question 1: Effects of Noise (Experiment 1)

Based on your high-noise experiment:

- How did noise affect accuracy and confidence?
- Why does noise make consensus harder? (Think about what each column "sees")
- Did the system still work? What does this suggest about robustness?

Question 2: Number of Columns (Experiment 2)

Based on your many-columns experiment:

- Did more columns improve performance? How?
- The brain has ~150,000 cortical columns. What advantage might this provide?
- Are there any downsides to having more columns?

Question 3: Vote Weight Trade-off (Experiment 3)

Based on your vote weight experiment:

- What is the optimal vote_weight and why?
- What's wrong with vote_weight = 0.1? (columns don't listen to each other)
- What's wrong with vote_weight = 0.9? (columns ignore their own evidence)
- What general principle does this illustrate about balancing individual vs. collective information?

5.2 Connection to Neuroscience

Question 4: Distributed Representation

In your simulation, no single column "knows" the answer — consensus emerges from many columns.

- How does this relate to the Thousand Brains Theory?
- What would happen if you only had ONE column?
- Why might the brain use this distributed approach instead of a single "decision center"?

Question 5: Lateral Connections

The voting mechanism simulates lateral connections between cortical columns.

- What is the purpose of these connections in your simulation?
- What would happen if you set vote_weight = 0 (no connections)?
- In the real brain, why might columns need to communicate?

5.3 Limitations

Important: Be honest about what this simplified model does NOT capture.

Complete this comparison:

Our Simulation	Real Cortical Columns	Why It Matters
6-20 columns	~150,000 columns	(your answer)
Simple probability vectors	~100,000 neurons per column	(your answer)
All columns connected	Sparse local connections	(your answer)
No reference frames	Grid cell-based reference frames	(your answer)
Static input	Temporal sequences	(your answer)
Pre-defined objects	Learns continuously	(your answer)

Question 6: Model Validity

- What aspects of real cortical processing does this simulation capture well?
- What important aspects does it completely miss?
- Would you trust this simulation to make predictions about the brain? Why or why not?

Part 6: Troubleshooting

6.1 Common Errors

"NameError: name 'evidence' is not defined"

Problem: You're using a variable before creating it.

Solution: Run cells in order from the top. Each step depends on previous steps.

"IndexError: index out of bounds"

Problem: Array index is wrong.

Solution: Check that true_object is 0-4 (Python uses 0-based indexing). Wolfram uses 1-5.

Columns all show 0% or 100% confidence

Problem: Normalization is wrong.

Solution: Make sure you divide by row_sums with axis=1 and keepdims=True.

Columns don't converge (stuck at low agreement)

Problem: Voting mechanism isn't working.

Solution: Check that you're combining own_contribution + vote_contribution correctly.

Wrong object wins

Problem: Signal isn't being added correctly.

Solution: In create_sensory_input, make sure you add signal to evidence[col, true_object], not a different position.

Plots don't appear

Problem: Matplotlib issue.

Solution: In Colab, plots should appear automatically. Try adding plt.show() at the end.

6.2 Debugging Tips

- **Print intermediate values:** Add print statements to see what's happening inside functions.
- **Check shapes:** Use print(array.shape) to verify array dimensions.
- **Check ranges:** Use print(array.min(), array.max()) to verify values are reasonable.
- **Test incrementally:** Test each function separately before combining them.
- **Reset and rerun:** If confused, restart the kernel and run all cells from the beginning.

6.3 Getting Help

20. Check this troubleshooting guide first

21. Post in the Module 02 Discussion Board with:

- Your code (the relevant function)
- The exact error message
- What you've already tried

22. Attend office hours (see course schedule)

23. Email the instructor with your notebook attached

Part 7: Submission Requirements

7.1 File Naming

FILE NAMING CONVENTION

Main Notebook:

L02_FirstName_LastName_ITAI4374

Examples:

L02_Maria_Garcia_ITAI4374.ipynb

L02_Maria_Garcia_ITAI4374.nb

If submitting Analysis separately:

L02a_FirstName_LastName_ITAI4374.pdf

7.2 What to Submit

Option A: Single File (Preferred)

Include everything in your notebook:

- All code cells with visible outputs
- All experiment results
- Visualization plots
- Analysis answers (in markdown cells at the end)

Option B: Two Files

If you prefer to write analysis separately:

24. **L02_...** — Code notebook with outputs
25. **L02a_...** — Analysis document (PDF, 1-2 pages)

7.3 Grading Rubric

Component	Points	What We're Looking For
Working Code	40	All functions implemented correctly, code runs without errors
Experiments	20	All 3 experiments completed with recorded observations
Visualization	15	Clear consensus plot showing convergence
Analysis	25	Thoughtful answers showing understanding of concepts
TOTAL	100	
Bonus: Extension	+10	One extension challenge completed (optional)

7.4 Submission Checklist

Before submitting, verify:

- File named correctly (L02_FirstName_LastName_ITAI4374)
- All code cells have been run (outputs visible)
- `create_sensory_input()` works correctly
- `initialize_beliefs()` works correctly
- `voting_round()` works correctly
- `run_voting()` works correctly
- `visualize_consensus()` produces readable plots
- All 3 experiments completed
- All analysis questions answered

Submit via Canvas by the deadline posted in the course schedule.

Good luck! Remember: the goal is understanding, not just getting code to work.